Отчет по лабораторной работе 3

Реализованы функции transpose_rows, transpose_cols, transpose_blocked, transpose_rows_omp, transpose_blocked_omp и дополнительно транспонирование на месте (inplace, inplace_omp — обмен блоков (ii, jj) ↔ (jj, ii), schedule(dynamic, 1) из-за треугольной нагрузки). Вариант D: double, $N_v = 4500$. Каждый запуск сверяется с transpose_rows поэлементно — во всех конфигурациях check=OK. Время — медиана 5 запусков после прогрева (omp_get_wtime() в MinGW тикает раз в 1 мс, поэтому малые матрицы транспонировались в цикле не менее 0.1 с за замер).

Система: Intel Core i5-9400F, 6 ядер / 6 потоков, 2.9 ГГц (до 4.1 ГГц); L1d 32 КБ и L2 256 КБ на ядро, L3 9 МБ общий; ОЗУ 2×16 ГБ DDR4-2666, 2 канала (пик ≈ 42.7 ГБ/с); Windows 10, GCC 16.2.0 (MSYS2 UCRT64), флаги -O2 -fopenmp.

Эксперимент 1 (1 поток)

N	Время rows, с	Время cols, с	BW rows, ГБ/с	BW cols, ГБ/с
256	0.000223	0.000108	4.71	9.67
512	0.000820	0.000449	5.11	9.34
1024	0.005750	0.003344	2.92	5.02
2048	0.032750	0.039250	2.05	1.71
4500	0.199000	0.210000	1.63	1.54

Эксперимент 2 (1 поток, N = 4500)

Блок b	Время, с	BW, ГБ/с
4	0.050500	6.42
8	0.036000	9.00
16 = b*	0.032750	9.89
32	0.035000	9.26
64	0.040750	7.95
128	0.033250	9.74

Эксперимент 3 (N = 4500, b* = 16, $T_1 = 0.199$ с)

Потоки	Время rows_omp, с	S rows_omp	Время blocked_omp, с	S blocked_omp
1	0.201000	0.99	0.032500	6.12
2	0.124000	1.60	0.021500	9.26
4	0.051000	3.90	0.022750	8.75
8	0.046000	4.33	0.023500	8.47
16	0.042750	4.66	0.024250	8.21

Эксперимент 4 (8 потоков, rows_omp)

Стратегия	Время, с	BW, ГБ/с
schedule(static)	0.048000	6.75
schedule(static, 1)	0.079000	4.10
schedule(static, 8)	0.053000	6.11

Дополнительно (in-place, b = 16): inplace (1 поток) — 0.0276 с против 0.0328 с у blocked; на 6 потоках inplace_omp — 0.0140 с (23.1 ГБ/с) против 0.0243 с (13.4 ГБ/с) у blocked_omp, на 8 потоках — 0.0146 с против 0.0263 с. Версия на месте быстрее в ~1.7 раза: она работает с одной матрицей (162 МБ вместо 324 МБ) и не требует чтения строк B перед записью (write-allocate).

Вывод:

В transpose_rows чтение A последовательное, а запись в B идёт с шагом N·8 байт: на каждый записанный double процессор загружает и затем вытесняет целую 64-байтную грязную кеш-линию. В transpose_cols последовательна запись, а с шагом идёт чтение, которое дешевле (нет обратной записи, работает предвыборка). Поэтому на малых N, когда данные в основном лежат в кешах (при N = 512 обе матрицы — 4 МБ, меньше L3), cols быстрее в 1.7–2 раза, и разница заметна до N = 1024. При N = 2048 шаг 16 КБ — степень двойки, все элементы столбца попадают в одни и те же наборы кеша (конфликтные промахи), и cols становится медленнее; при N = 4500 обе матрицы (по 162 МБ) намного больше L3, обе версии упираются в задержку DRAM (одно полезное число из 8 в линии) и работают одинаково медленно (~1.6 ГБ/с). Блочная версия даёт ускорение в 6 раз: при b* = 16 рабочий набор двух блоков $2 \cdot 16^2 \cdot 8$ Б = 4 КБ и 32 строки/страницы с запасом помещаются в L1d (32 КБ) и L1 DTLB (64 записи); при b = 64 набор (64 КБ) уже больше L1 — время растёт, а при b = 128 (256 КБ) блоки живут в L2 (256 КБ), и время снова близко к лучшему. Ускорение ограничено сильнее, чем в ЛР2: транспонирование не выполняет вычислений (арифметическая интенсивность 0), и все потоки делят одну шину памяти — blocked_omp выходит на потолок ~15 ГБ/с (с учётом write-allocate ≈ 22 ГБ/с реального трафика при пике 42.7 ГБ/с) уже на 2 потоках, и дальнейшее добавление потоков ничего не даёт, а интеграл из ЛР2 ограничен только числом ядер. rows_omp масштабируется лучше (до 4.7), потому что у неё узкое место — задержка промахов, а не полоса, и несколько ядер перекрывают промахи друг друга. schedule(static, 1) медленнее schedule(static) в 1.65 раза из-за false sharing: соседние строки i и i+1 достаются разным потокам, а они пишут в соседние столбцы B, т.е. в одну кеш-линию, и линия постоянно перебрасывается между ядрами протоколом когерентности. При static каждый поток пишет в свой непрерывный диапазон из ~560 столбцов, и общие линии есть только на границах; static, 8 почти устраняет проблему, но из-за невыровненных строк (36000 Б не кратно 64) часть линий всё равно делится.